

TEAVS–ADCRS Core Integration Interface Specification

1. Purpose

This document defines the interface used to submit TEAVS data into the PHOENIX core-model integration workflow.

The primary endpoint is:

POST /api/ingestion/core

The endpoint receives a TEAVS event payload, applies security controls at the API Gateway, and forwards accepted requests to the downstream integration workflow through RabbitMQ.

2. Integration Flow

The verified flow is:

TEAVS → API Gateway → RabbitMQ → data-ingestion-service → coreModelIntegration() → model inference

The current local environment successfully reaches coreModelIntegration().

Full inference remains dependent on the availability of the configured trained model artifact.

3. Endpoint

Method:

POST

Path:

/api/ingestion/core

Content-Type:

application/json

Authentication:

Bearer JWT

Example header:

Authorization: Bearer <access_token>

4. Authorised Roles

Access to the endpoint is restricted using role-based access control.

The permitted roles are:

- admin
- ingestion_service

Other authenticated roles such as:

- analyst
- end_user

are not permitted to access the endpoint.

An authenticated user without an approved role receives:

HTTP 403 Forbidden

5. Request Payload

The endpoint uses the current CoreModelIntegrationPayload interface.

The required structure is:

```
{
  "url": "https://example.com/alert",
  "text": "Potential phishing activity detected",
  "timestamp": "2026-09-27T09:50:00+10:00",
  "hazard_type": "phishing",
  "hazard_severity": 4,
  "hazard_timestamp": "2026-09-27T09:49:00+10:00",
  "hazard_location": "VIC",
  "hazard_status": "active",
  "alert_level": "critical",
  "source": "TEAVS"
}
```

Field Definitions

Field	Type	Required	Description
url	string	Yes	Source or reference URL associated with the TEAVS event
text	string	Yes	Event or alert description
timestamp	string	Yes	Timestamp associated with the submitted integration request
hazard_type	string	Yes	Type of detected hazard
hazard_severity	number	Yes	Numeric hazard severity
hazard_timestamp	string	Yes	Time at which the hazard was detected
hazard_location	string	Yes	Location associated with the event
hazard_status	string	Yes	Current hazard status
alert_level	string	Yes	Alert classification or level
source	string	Yes	Originating system, for example TEAVS

6. Request Processing Order

The endpoint currently applies the following controls before forwarding the request:

JWT Authentication

↓

Role-Based Access Control

↓

Rate Limiting

↓

Input Validation

↓

coreModelIntegration Controller

↓

RabbitMQ

This ensures that invalid or unauthorised requests are rejected before entering the downstream model-processing workflow.

7. Success Response

A valid request submitted by an authorised role returns:

HTTP 202 Accepted

Example:

```
{  
  "status": 202,  
  "message": "Core model integration data sent successfully"  
}
```

A 202 response confirms that the request was accepted for downstream processing.

It does not confirm that model inference has completed successfully.

8. Authentication Errors

Missing JWT

Response:

HTTP 401 Unauthorized

Example:

```
{  
  "status": 401,  
  "message": "No token provided"  
}
```

Invalid JWT

Response:

HTTP 401 Unauthorized

Example:

```
{  
  "status": 401,  
  "message": "Invalid token"  
}
```

9. Authorisation Error

An authenticated user with a role that is not permitted to use the endpoint receives:

HTTP 403 Forbidden

Example:

```
{  
  "status": 403,  
  "message": "Access denied"  
}
```

10. Validation Errors

Incomplete or invalid payloads are rejected before they reach the core integration controller.

Typical response:

HTTP 400 Bad Request

Examples include:

- missing required fields
- incorrect field types
- malformed JSON

The validation control was verified through both manual testing and the automated TEAVS–ADCRS security tester.

11. Rate Limiting

The endpoint includes request-rate protection.

Current local configuration:

20 requests per 60-second window per client IP address

Requests above the configured threshold receive:

HTTP 429 Too Many Requests

Example response:

```
{  
  "status": 429,  
  "message": "Rate limit exceeded"  
}
```

The current limiter uses an in-memory counter.

For a distributed production environment, a shared store such as Redis would provide stronger cross-instance enforcement.

12. Dependency Failure

If RabbitMQ is unavailable when the request is submitted, the API Gateway does not return 202 Accepted.

The verified failure response was:

HTTP 500 Internal Server Error

Example:

```
{  
  "status": 500,  
  "message": "Error integrating core model"  
}
```

A future improvement could use 503 Service Unavailable for dependency outages.

13. RabbitMQ Integration

Accepted requests are published to:

`core_model_integration_queue`

The data-ingestion-service consumes messages from this queue and passes them to:

`coreModelIntegration()`

Runtime verification confirmed that a TEAVS test payload successfully reached the queue consumer.

14. Downstream Processing

The downstream service:

1. creates an integration log
2. receives the submitted payload
3. locates the configured training model
4. invokes model inference
5. stores the model output and final status

The configured model filename is:

`final_core_xgb_xgboost_trey_xgb_core_v2_epoch_100.joblib`

During local verification, processing reached the model lookup stage successfully.

15. Current Integration Limitation

The trained model artifact was not available in the local repository or local model storage.

As a result, the downstream process returned:

Training model not found

The integration has therefore been verified successfully up to the model-loading stage.

Full inference testing requires the approved trained model artifact to be made available to the backend.

16. Duplicate Submission Behaviour

Submitting the same valid payload twice currently results in both requests being accepted with:

HTTP 202 Accepted

No duplicate or idempotency control was identified in the inspected API Gateway, RabbitMQ consumer, or coreModelIntegration() path.

A future implementation should use a stable TEAVS event identifier or idempotency key to prevent accidental repeated processing.

17. Security Controls Summary

Control	Current Status
JWT authentication	Implemented and verified
Role-based access control	Implemented and verified
Payload validation	Implemented and verified
Malformed JSON handling	Verified
Rate limiting	Implemented and verified
RabbitMQ failure handling	Verified
Duplicate detection	Not currently implemented
RabbitMQ message delivery	Verified
Downstream core processing	Verified to model-loading stage
Final model inference	Blocked by missing model artifact

18. Handover Notes

The endpoint is currently suitable for continued TEAVS–ADCRS integration testing within the local PHOENIX environment.

The main remaining dependency for full end-to-end verification is the trained model artifact required by the downstream core-model service.

Future work should also consider adding a stable event identifier for idempotency and moving rate limiting to a shared Redis-backed implementation for production deployment.